

Texas Hold'em Simulator

Project Description

The goal of our project is to create an interactive player-versus-computer Texas Hold'em Poker game using Python. As students interested in programming and game development, the motivation behind our project is to explore how algorithms, probability, and decision-making can be implemented in a realistic and engaging card game. Poker is a game that combines chance, strategy, and critical thinking, making it an excellent opportunity to apply programming concepts while creating an entertaining experience for users. Through this project, we aim to develop a game that not only allows players to enjoy a classic card game but also demonstrates how computers can make intelligent decisions based on available information.

The game begins by creating and shuffling a standard 52-card deck, then dealing two private cards to both the player and the computer, along with five community cards that are revealed throughout the game. Each player starts with a fixed amount of virtual chips, and blinds are used to create an initial pot before cards are dealt. During each round, the user can choose to fold, call, or raise, while the computer evaluates its hand strength and estimates its probability of winning before making its own decision. To make gameplay more realistic, the computer can occasionally bluff by making aggressive moves even when holding weaker cards. At the end of each hand, the game evaluates the best possible five-card poker hand that each participant can make using their private cards and the community cards, determines the winner, and awards the pot accordingly.

The project involves creating functions to generate and manage the deck, deal cards, evaluate poker hands, handle betting rounds, track chip counts, and control the computer's decision-making process. Additional features include a hand-ranking system that recognizes combinations such as pairs, straights, flushes, and full houses, as well as a probability-based artificial intelligence system that allows the computer to make strategic decisions. Overall, this project combines object-oriented programming, probability, and game logic to create an

engaging poker experience that showcases the practical applications of Python programming.

Team Member Names and Contributions

All team members worked together throughout the planning and development of the project, scheduling outside meetings to discuss game mechanics, function design, and implementation. Both members contributed to testing, debugging, and refining the overall poker game experience. The project idea originated from combining interests in programming, probability, and strategy games, with each team member contributing unique ideas that shaped the final product.

- **Micha Gilman:** Proposed the idea of creating a player-versus-computer poker game, which became the foundation of the project. Micha was responsible for developing the first five functions of the program: `create_deck()`, `deal_cards()`, `show_game()`, `evaluate_hand()`, and `calculate_win_probability()`. These functions handle the creation and management of the deck, card distribution, displaying game information, evaluating poker hands, and calculating the probability of winning based on the cards available. Micha also contributed to testing and refining the game mechanics to ensure accurate hand evaluation and gameplay.

- **Henrique Bonazzi:** Proposed that the project focus on a strategy-based game involving probability and decision-making, helping guide the overall direction of the project. Henrique was responsible for developing the last five functions of the program: `computer_decision()`, `place_blinds()`, `betting_round()`, `determine_winner()`, and `play_hand()`. These functions manage the computer opponent's actions, betting system, pot creation, winner determination, and the overall flow of each poker hand. Henrique also contributed to designing the computer opponent's decision-making system, including betting, folding, calling, raising, and bluffing behaviors.

- **Micha Gilman and Henrique Bonazzi:** Collaboratively developed the `main()` function, which serves as the central game loop and connects all other functions together. Both team members worked together on debugging, testing gameplay

scenarios, integrating individual functions, and ensuring the program operated smoothly from start to finish.

Project Code

This section contains the main project code for our player-versus-computer poker game. The code is organized into functions that create the deck, deal cards, evaluate hands, control betting, make computer decisions, determine the winner, and run the full game.

Setup

The following modules are imported to help shuffle cards, generate combinations of hands, and add random computer behavior.

```
import random
import itertools
```

Project Code continued...

```
# Creates a standard 52-card deck
```

```
def create_deck():
    suits = ["♠", "♥", "♦", "♣"]
    ranks = ["2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A"]
    deck = [rank + suit for suit in suits for rank in ranks]
    random.shuffle(deck)
    return deck
```

```
# Deals two cards to the player, two cards to the computer, and five community cards
```

```
def deal_cards(deck):
    player_hand = [deck.pop(), deck.pop()]
    computer_hand = [deck.pop(), deck.pop()]
    table_cards = [deck.pop() for _ in range(5)]
```

```
return player_hand, computer_hand, table_cards
```

```
# Displays the player's cards and the community cards
```

```
def show_game(player_hand, table_cards):  
    print("Your hand:", player_hand)  
    print("Community cards:", table_cards)
```

```
# Evaluates the strength of a poker hand
```

```
def evaluate_hand(cards):  
    ranks_order = {  
        "2": 2, "3": 3, "4": 4, "5": 5, "6": 6, "7": 7,  
        "8": 8, "9": 9, "10": 10, "J": 11, "Q": 12, "K": 13, "A": 14  
    }  
  
    ranks = [card[:-1] for card in cards]  
    suits = [card[-1] for card in cards]  
    values = sorted([ranks_order[rank] for rank in ranks], reverse=True)  
  
    rank_counts = {}  
    for value in values:  
        rank_counts[value] = rank_counts.get(value, 0) + 1  
  
    flush = len(set(suits)) == 1  
    straight = len(set(values)) == 5 and max(values) - min(values) == 4  
  
    if straight and flush:  
        return 9  
    elif 4 in rank_counts.values():  
        return 8  
    elif 3 in rank_counts.values() and 2 in rank_counts.values():  
        return 7  
    elif flush:  
        return 6  
    elif straight:  
        return 5
```

```
elif 3 in rank_counts.values():
    return 4
elif list(rank_counts.values()).count(2) == 2:
    return 3
elif 2 in rank_counts.values():
    return 2
else:
    return 1
```

Finds the best possible hand from seven cards

```
def best_hand_score(cards):
    possible_hands = itertools.combinations(cards, 5)
    best_score = 0

    for hand in possible_hands:
        score = evaluate_hand(list(hand))
        if score > best_score:
            best_score = score

    return best_score
```

Computer decides whether to fold, call, or raise

```
def computer_decision(hand_strength):
    if hand_strength >= 6:
        return "raise"
    elif hand_strength >= 3:
        return "call"
    else:
        if random.random() < 0.15:
            return "raise"
        return "fold"
```

Creates the starting pot using blinds

```
def place_blinds(player_chips, computer_chips):
```

```
small_blind = 5
big_blind = 10
```

```
player_chips -= small_blind
computer_chips -= big_blind
pot = small_blind + big_blind
```

```
return player_chips, computer_chips, pot
```

```
# Handles a simple betting round
```

```
def betting_round(player_chips, computer_chips, pot, computer_strength):
    print("Current pot:", pot)
    player_move = input("Would you like to fold, call, or raise? ").lower()
```

```
    if player_move == "fold":
        return player_chips, computer_chips + pot, 0, "computer"
```

```
    elif player_move == "call":
        bet = 10
        player_chips -= bet
        computer_chips -= bet
        pot += bet * 2
```

```
    elif player_move == "raise":
        bet = 20
        player_chips -= bet
        pot += bet
```

```
    computer_move = computer_decision(computer_strength)
    print("Computer chooses to:", computer_move)
```

```
    if computer_move == "fold":
        return player_chips + pot, computer_chips, 0, "player"
    elif computer_move == "call":
        computer_chips -= bet
        pot += bet
```

```

        elif computer_move == "raise":
            computer_chips -= bet
            pot += bet

    return player_chips, computer_chips, pot, "continue"

# Determines who wins the hand
def determine_winner(player_hand, computer_hand, table_cards):
    player_score = best_hand_score(player_hand + table_cards)
    computer_score = best_hand_score(computer_hand + table_cards)

    print("Your final hand:", player_hand)
    print("Computer final hand:", computer_hand)
    print("Community cards:", table_cards)

    if player_score > computer_score:
        print("You win the hand!")
        return "player"
    elif computer_score > player_score:
        print("Computer wins the hand!")
        return "computer"
    else:
        print("It is a tie!")
        return "tie"

# Runs one full hand of poker
def play_hand(player_chips, computer_chips):
    deck = create_deck()
    player_hand, computer_hand, table_cards = deal_cards(deck)

    player_chips, computer_chips, pot = place_blinds(player_chips,
computer_chips)

    print("\nNew hand begins!")
    show_game(player_hand, table_cards)

```

```
computer_strength = best_hand_score(computer_hand + table_cards)
```

```
player_chips, computer_chips, pot, status = betting_round(  
    player_chips, computer_chips, pot, computer_strength  
)
```

```
if status == "player":  
    print("Computer folded. You win the pot!")  
    return player_chips, computer_chips
```

```
elif status == "computer":  
    print("You folded. Computer wins the pot!")  
    return player_chips, computer_chips
```

```
winner = determine_winner(player_hand, computer_hand, table_cards)
```

```
if winner == "player":  
    player_chips += pot  
elif winner == "computer":  
    computer_chips += pot  
else:  
    player_chips += pot // 2  
    computer_chips += pot // 2
```

```
return player_chips, computer_chips
```

```
# Main function that runs the full game
```

```
def main():
```

```
    player_chips = 1000  
    computer_chips = 1000  
    playing = True
```

```
    print("Welcome to Player vs Computer Poker!")
```

```
    while playing and player_chips > 0 and computer_chips > 0:
```

```
print("\nYour chips:", player_chips)
print("Computer chips:", computer_chips)

player_chips, computer_chips = play_hand(player_chips, computer_chips)

replay = input("Would you like to play another hand? yes/no: ").lower()

if replay != "yes":
    playing = False

print("Thanks for playing!")

main()
```

Playing Game Demo...

```
=====
ROUND 1
=====
```

Your Money: \$990
Computer Money: \$990
Pot: \$20

Your Hand:
['AC', '5S', '5D', '2H', 'KC']
Hand Type: One Pair

I fold.

You win the pot of \$20.
Bankrolls -> You: \$1010 | Computer: \$990

Press Enter for next round or q to quit:

=====

ROUND 2

=====

Your Money: \$1000

Computer Money: \$980

Pot: \$20

Your Hand:

['KH', '6C', 'JD', '10C', '6S']

Hand Type: One Pair

I fold.

You win the pot of \$20.

Bankrolls -> You: \$1020 | Computer: \$980

Press Enter for next round or q to quit:

=====

ROUND 3

=====

Your Money: \$1010

Computer Money: \$970

Pot: \$20

Your Hand:

['QC', '7H', '6H', '8C', '4C']

Hand Type: High Card

"I have Two Pair"

Your move (f=fold, c=call, r=raise): c

Enter call amount: \$10

I call.

=====
SHOWDOWN
=====

Human Hand:
['QC', '7H', '6H', '8C', '4C']
Hand Type: High Card

Computer Hand:
['QH', 'QS', '3H', 'KS', '9C']
Hand Type: One Pair

Computer wins!

Bankrolls -> You: \$1000 | Computer: \$1000

Press Enter for next round or q to quit:

=====
ROUND 4
=====

Your Money: \$990
Computer Money: \$990
Pot: \$20

Your Hand:
['AD', '3C', '7S', '9S', '8D']
Hand Type: High Card

"I have Three of a Kind"

Your move (f=fold, c=call, r=raise): f

I call.

Computer wins the pot of \$20.

Bankrolls -> You: \$990 | Computer: \$1010

Press Enter for next round or q to quit:

=====

ROUND 5

=====

Your Money: \$980

Computer Money: \$1000

Pot: \$20

Your Hand:

['9H', '7D', '7C', '9D', '8S']

Hand Type: Two Pair

I fold.

You win the pot of \$20.

Bankrolls -> You: \$1000 | Computer: \$1000

Press Enter for next round or q to quit:

--- DECK RESHUFFLED ---

=====

ROUND 6

=====

Your Money: \$990

Computer Money: \$990

Pot: \$20

Your Hand:

['2C', '3D', '2S', '5H', '10C']

Hand Type: One Pair

I fold.

You win the pot of \$20.

Bankrolls -> You: \$1010 | Computer: \$990

Press Enter for next round or q to quit:

=====

ROUND 7

=====

Your Money: \$1000

Computer Money: \$980

Pot: \$20

Your Hand:

['6H', '7C', '3C', 'KC', '8C']

Hand Type: High Card

"I have Two Pair"

Your move (f=fold, c=call, r=raise): c

Enter call amount: \$50

I call.

=====

SHOWDOWN

=====

Human Hand:

['6H', '7C', '3C', 'KC', '8C']

Hand Type: High Card

Computer Hand:
['5C', 'KH', 'QH', '9H', 'QS']
Hand Type: One Pair

Computer wins!

Bankrolls -> You: \$950 | Computer: \$1050

Press Enter for next round or q to quit:

=====
ROUND 8
=====

Your Money: \$940
Computer Money: \$1040
Pot: \$20

Your Hand:
['7H', '4C', 'QC', '7D', '6S']
Hand Type: One Pair

"I have a Straight Flush"

Your move (f=fold, c=call, r=raise): c
Enter call amount: \$50

I call.

=====
SHOWDOWN
=====

Human Hand:
['7H', '4C', 'QC', '7D', '6S']
Hand Type: One Pair

Computer Hand:
['2D', '4D', '3S', '6C', '9D']
Hand Type: High Card

Human wins!

Bankrolls -> You: \$1010 | Computer: \$990

Press Enter for next round or q to quit: ^A

Reflection

Henrique Bonazzi: Throughout this project, I was able to combine my interest in poker with my growing knowledge of Python programming. Since I have always enjoyed playing poker, creating a poker game gave me the opportunity to apply concepts from the game while also developing my programming skills. Working on this project helped me improve my teamwork abilities, particularly through the use of GitHub, where my partner and I collaborated on the code simultaneously and learned how to manage changes effectively. I also gained a better understanding of how to create and organize functions, which taught me the importance of breaking large programs into smaller, manageable sections to improve readability, organization, and overall program flow. Most importantly, this project increased my enjoyment of Python because it allowed me to learn new concepts and immediately apply them to create something that I found personally interesting and rewarding.

Micha Gilman:

My skills in poker are not great, but I do find the game enjoyable, so I was enthusiastic to create a code that could help me and others learn how to play the game better in a low stakes environment. Having an interactive

game was a unique and engaging way to work with Python that motivated me to continuously improve the code for the best and most user friendly experience. I learned how to use platforms in new ways such as collaborating with my team on GitHub and learned solid communication skills to achieve an efficient workflow. Working in a team was beneficial because my partner could identify bugs or errors that I wouldn't normally be able to see and I was able to receive feedback to enhance my skills. This project helped me cement my familiarity with functions and methods and opened my eyes to new career opportunities and interests.